408 计算机网络：传输层复习笔记

传输层是 408 计算机网络中的**核心重点章节**，尤其是 TCP 的可靠传输、滑动窗口、流量控制、拥塞控制和三次握手、四次挥手。

相较于前面的网络层，传输层计算题的公式并不算多，但很容易出现**多个机制结合的综合题**，例如：

- 根据 TCP 序号和确认号判断传输了多少字节。
- 根据接收窗口与拥塞窗口判断实际发送窗口。
- 根据超时或重复 ACK 判断拥塞窗口变化。
- 根据 TCP 三次握手、四次挥手计算 RTT。
- 根据 TCP 报文段发送过程判断累计确认号。

这份笔记继续采用之前的结构：**基本概念 → 协议原理 → 计算公式 → 典型例题 → 易错点 → 考前速记**。

一、传输层的基本概念

1. 传输层的作用

传输层位于网络层之上、应用层之下。



传输层的核心任务：为不同主机上的应用进程提供端到端的逻辑通信。

注意区分：

层次	通信对象
数据链路层	相邻节点
网络层	主机到主机
传输层	进程到进程

例如：

你的计算机同时运行：

- 浏览器。
- QQ。
- 微信。
- SSH 客户端。

网络层将 IP 数据报交付到你的计算机后，传输层需要进一步判断：**这些数据究竟应该交给哪个应用程序？**

这就需要端口号。

2. 传输层的主要功能

1. 进程间通信。
2. 复用和分用。
3. 差错检测。
4. 根据协议提供可靠或不可靠的数据传输服务。

5. TCP 的流量控制与拥塞控制。
6. TCP 的连接管理。

3. TCP 与 UDP

传输层最重要的两个协议：

- TCP：传输控制协议。
- UDP：用户数据报协议。

特性	TCP	UDP
连接方式	面向连接	无连接
可靠性	提供可靠字节流服务	不保证可靠交付
数据组织	面向字节流	面向报文
顺序保证	保证按序交付字节流	不保证
流量控制	有	无内建机制
拥塞控制	有	无内建机制
首部长度	20～60 B	固定 8 B
广播/组播	不支持	可以用于广播或组播
典型应用	HTTP/1.1、SSH、FTP	DNS、DHCP、实时音视频等

注意：HTTP/3 使用 QUIC，而 QUIC 运行在 UDP 之上。所以不能机械地认为所有 HTTP 都直接使用 TCP。

二、端口号与套接字（必考）

1. 端口号

端口号用于标识主机上的通信端点，帮助传输层将数据交给正确的应用进程。

TCP、UDP 端口号长度均为：

$$\boxed{16\ \text{bit}}$$

范围：

$$\boxed{0 \sim 65535}$$

常见分类：

类别	范围	用途
系统端口	0～1023	常见标准服务
用户端口	1024～49151	注册或分配给应用服务
动态/私有端口	49152～65535	常用于临时端口

这里是 IANA 的分类范围，具体操作系统的临时端口分配范围可能不同。

2. 必背常见端口

协议	传输层	端口
FTP 控制连接	TCP	21
FTP 默认主动模式数据连接	TCP	20
SSH	TCP	22
Telnet	TCP	23
SMTP	TCP	25
DNS	UDP/TCP	53
DHCPv4 服务器	UDP	67
DHCPv4 客户端	UDP	68
HTTP	TCP	80
POP3	TCP	110
IMAP	TCP	143
HTTPS（HTTP/1.1、HTTP/2）	TCP	443
HTTPS（HTTP/3）	UDP	443
BGP	TCP	179

补充：FTP 的数据连接并非永远使用服务器端口 20；被动模式可采用其他端口。

3. 套接字 Socket

传统网络编程中，套接字通信端点通常由：

$$\boxed{\text{IP 地址} + \text{端口号}}$$

表示。

例如：

`192.168.1.10:50000`

其中：

- IP 地址：标识网络接口。
- 端口号：标识通信端点。

但需要注意，实际操作系统中的 Socket 是一个编程抽象，不只是两个数字。

4. TCP 连接由什么确定?

一个 TCP 连接可以由以下四元组唯一标识：

$$\boxed{(\text{源 IP}, \text{源端口}, \text{目的 IP}, \text{目的端口})}$$

例如：

连接 1：

`192.168.1.10:50001 → 203.0.113.10:80`

连接 2：

`192.168.1.10:50002 → 203.0.113.10:80`

虽然访问的是同一台服务器、同一个端口，但由于源端口不同，这是两个不同的 TCP 连接。

408 高频考点：同一个服务器端口可以同时服务多个 TCP 连接。

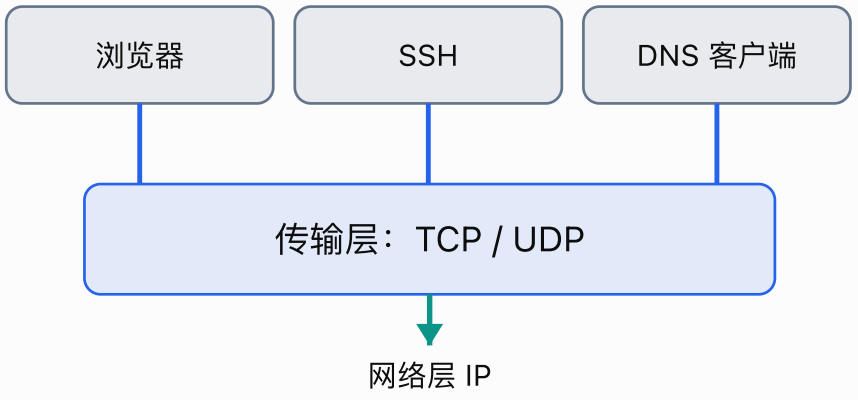
三、复用与分用

1. 复用

发送方将多个应用进程的数据交给传输层，由传输层通过 TCP/UDP 进行发送。

2. 分用

接收方传输层根据报文中的端口等信息，将数据交给相应应用。



发送时进行复用，接收时进行分用。

3. TCP 与 UDP 的分用区别

UDP 通常根据本地目的 IP、目的端口等信息选择接收 Socket。

TCP 则需要根据连接四元组区分不同连接。

重要结论：TCP 服务器可以在同一个监听端口上同时建立多个客户端连接。

四、UDP 用户数据报协议（重点）

1. UDP 的主要特点

UDP：User Datagram Protocol。

特点：

- 无连接。
- 尽最大努力交付。
- 不保证可靠性。
- 面向报文。
- 首部开销较小。
- 不提供 TCP 那样的流量控制和拥塞控制。
- 支持一对一、一对多等通信模式。

UDP 不等于一定传输速度比 TCP 快。

UDP 机制简单、首部开销低，但实际传输性能还取决于网络环境、应用设计、数据量等因素。

2. 面向报文是什么意思?

假设应用程序向 UDP 发送：

- 第一条消息：100 B。
- 第二条消息：200 B。

UDP 会分别封装成两个用户数据报。

接收时保留报文边界。

这与 TCP 面向字节流不同。

3. UDP 首部格式

UDP 首部固定：

$$\boxed{8\ \text{Byte}}$$



四个字段：

1. 源端口：16 bit。
2. 目的端口：16 bit。
3. 长度：16 bit。
4. 校验和：16 bit。

4. UDP 长度字段（常考）

UDP 长度字段表示：

$$\boxed{\text{UDP 长度} = \text{UDP 首部} + \text{UDP 数据}}$$

例如：

UDP 数据部分为 1000 B。

则 UDP 长度：

$$1000 + 8 = 1008\ \text{B}$$

$$\boxed{1008\ \text{B}}$$

不要忘记加上 8 B 首部。

5. UDP 校验和

UDP 校验和用于检测传输过程中发生的错误。

计算时涉及：

- UDP 伪首部。
- UDP 首部。
- UDP 数据。

伪首部中的主要信息包括源和目的 IP 地址以及协议标识等。

注意：

伪首部只参与校验和计算，不是真正传输在 UDP 报文前面的首部。

在 IPv4 中，UDP 校验和可以为 0，表示未使用校验；在普通 IPv6 UDP 通信中，校验和通常是必须的。

6. UDP 最大数据长度

在传统 IPv4、无 IP 选项、无 IP 分片重组之外额外限制的理论情况下：

IPv4 数据报最大总长度：

$$65535\ \text{B}$$

IPv4 首部：

$$20\ \text{B}$$

UDP 首部：

$$8\ \mathrm{B}$$

所以最大 UDP 数据部分：

$$65535 - 20 - 8$$

$$\boxed{65507\ \mathrm{B}}$$

注意：这只是经典 IPv4 情形下的理论上限，实际网络还受到 MTU、应用和操作系统限制。

五、TCP 传输控制协议（必考）

TCP：Transmission Control Protocol。

TCP 的主要特点：

1. 面向连接。
2. 提供可靠交付。
3. 面向字节流。
4. 支持全双工通信。
5. 提供流量控制。
6. 提供拥塞控制。

1. 面向字节流

TCP 将应用程序交付的数据看作连续的字节序列。

例如应用程序连续写入：

`ABC`

`DEF`

对 TCP 来说就是一个连续字节流：

`ABCDEF`

接收方可能一次读取：

`AB`

再读取：

`CDEF`

也可能一次全部读取。

TCP 不保留应用层写入时的消息边界。

因此开发应用层协议时，通常需要额外定义：

- 固定长度消息。
- 长度字段。
- 分隔符。
- 或其他消息边界机制。

2. TCP 可靠传输依赖什么？

TCP 通过多个机制共同实现可靠字节流服务：

- 序号。
- 确认号。
- 校验和。
- 超时重传。

- 重复确认。
- 接收缓存与按序交付。
- 滑动窗口。

注意：TCP 的可靠传输不是只依靠 ACK，也不是只依靠校验和。

六、TCP 报文段首部（必考）

TCP 首部最小为：

$$\boxed{20\ \text{Byte}}$$

传统 TCP 首部最大为：

$$\boxed{60\ \text{Byte}}$$

1. TCP 首部字段



重点字段：

字段	长度	作用
源端口	16 bit	发送端端口
目的端口	16 bit	接收端端口
序号	32 bit	当前数据的字节序号
确认号	32 bit	期望收到的下一个字节序号
数据偏移	4 bit	TCP 首部长度
控制标志	若干 bit	SYN、ACK、FIN 等
窗口	16 bit	通告接收窗口
校验和	16 bit	差错检测
紧急指针	16 bit	与 URG 等机制相关

2. TCP 数据偏移

数据偏移字段的单位是：

$$\boxed{4\ \text{Byte}}$$

如果数据偏移字段为 5：

$$5 \times 4 = 20\ \text{B}$$

如果为 8：

$$8 \times 4 = 32\ \text{B}$$

公式：

$$\boxed{\text{TCP 首部长度} = \text{数据偏移} \times 4}$$

3. TCP 控制标志

重点掌握：

标志	含义
SYN	建立连接、同步序号
ACK	确认号字段有效
FIN	发送方结束自己的发送方向
RST	复位连接
PSH	提示尽快向应用交付数据
URG	紧急指针字段有效

最重要的是 SYN、ACK、FIN。

七、TCP 序号与确认号（必考计算）

这是传输层最重要的基础计算题型。

1. TCP 序号 SEQ

TCP 对发送的字节流进行编号。

一个 TCP 数据报文段的 SEQ 通常表示：

该报文段数据部分第一个字节的序号。

例如：

发送方发送 1000 B 数据。

起始序号：

$$SEQ = 100$$

那么数据部分的序号范围：

$$100 \sim 1099$$

下一段新数据起始序号：

$$\boxed{1100}$$

公式：

$$\boxed{\text{下一段 SEQ} = \text{当前 SEQ} + \text{数据长度}}$$

这里默认报文段不包含消耗序号的 SYN、FIN 标志。

2. TCP 确认号 ACK

TCP 的确认号表示：

接收方期望收到的下一个字节序号。

例如：

接收方已连续收到序号：

$$100 \sim 1099$$

则确认号：

$$\boxed{ACK = 1100}$$

含义：

小于 1100 的所有相关字节都已经按序收到。

3. 典型例题

A 向 B 发送：

- SEQ=3000。
- 数据长度为 800 B。

B 正确收到后，应回复什么确认号?

数据范围：

$$3000 \sim 3799$$

下一字节：

$$3799 + 1 = 3800$$

因此：

$$\boxed{ACK = 3800}$$

考试快捷法：

$$\boxed{ACK = SEQ + \text{数据长度}}$$

前提是该段数据正好从接收方当前期待的序号开始，且连续正确接收。

八、TCP 累积确认（必考）

TCP 的基本确认方式是累积确认。

例如：

接收方依次收到：

- 1000～1499。
- 1500～1999。
- 2500～2999。

其中：

`2000～2499` 丢失。

虽然接收方收到了 `2500～2999`，但还有前面的字节缺失。

因此接收方可以继续发送：

$$\boxed{ACK = 2000}$$

而不是 ACK=3000。



1. 重复 ACK

如果后续不断收到失序数据，接收方可能重复发送相同确认号。

例如：

```
纯文本
ACK=2000
ACK=2000
ACK=2000
ACK=2000
```

发送方收到 3 个重复 ACK 后，经典 TCP Reno 通常触发快速重传。

注意：这里的 3 个重复 ACK 是在原来的正常 ACK 之外又收到 3 个相同 ACK。

2. SACK 选择确认

现代 TCP 还可以通过 SACK 选项告知发送方哪些非连续的数据块已经收到。

但要区分：

- 基本 ACK：累积确认。
- SACK：补充说明已收到的非连续字节范围。

在 408 常规题中，如果题目没有给出 SACK 相关条件，通常按照累积确认分析。

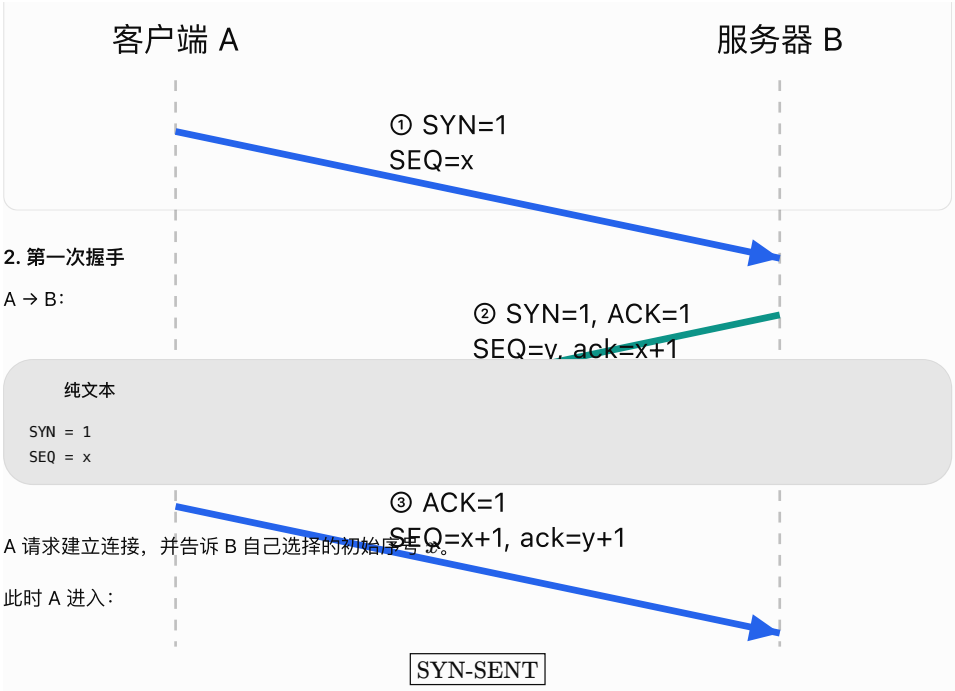
九、TCP 三次握手（必考重点）

TCP 建立连接通常采用三次握手。

目的：

- 确认双方具备基本通信能力。
- 同步双方初始序号。
- 协商 MSS、窗口扩大、SACK 等连接参数。

1. 三次握手过程



2. 第一次握手

A → B：

```
纯文本
SYN = 1
SEQ = x
```

A 请求建立连接，并告诉 B 自己选择的初始序号 x。

此时 A 进入：

$$\boxed{\text{SYN-SENT}}$$

三次握手示意：时间从上到下，A 主动发起连接，B 被动接受连接。

3. 第二次握手

B → A：

```
纯文本
SYN = 1
ACK = 1
SEQ = y
ack = x + 1
```

B 同意建立连接，并告知自己的初始序号 y。

B 进入：

$$\boxed{\text{SYN-RCVD}}$$

注意：

SYN 会消耗一个序号。

因此：

$$\boxed{ack = x + 1}$$

4. 第三次握手

A → B：

```
纯文本
ACK = 1
SEQ = x + 1
ack = y + 1
```

A 确认收到 B 的 SYN。

A 进入：

$$\boxed{\text{ESTABLISHED}}$$

B 收到第三次握手后，也进入 ESTABLISHED。

5. 为什么需要三次握手?

三次握手让双方确认彼此的连接请求和初始序号，并避免某些过期连接请求被错误地当成有效新连接。

可以记成：

1. A：我要建立连接，我的初始序号是 x。
2. B：收到，我的初始序号是 y，我确认 x。
3. A：我也收到了 y，并确认它。

6. 典型计算题

客户端发送：

```
纯文本
SYN=1, SEQ=500
```

服务器返回：

```
纯文本
SYN=1, ACK=1, SEQ=1000
```

问服务器的确认号，以及客户端第三次握手的序号和确认号。

第二次握手：

$$ack = 500 + 1 = 501$$

第三次握手：

$$SEQ = 501$$

$$ack = 1000 + 1 = 1001$$

答案：

$$\boxed{SEQ = 501, \quad ack = 1001}$$

7. 三次握手需要多少 RTT?

设一个 RTT 为往返时间。

在忽略报文发送、处理等开销时：

- SYN 发送出去。
- SYN+ACK 返回，经历约 1 RTT。
- 第三个 ACK 到达服务器，再经过约 0.5 RTT。

因此：

客户端收到第二次握手并发送第三次握手时，约经过 1 RTT。

服务器收到第三次握手、确认连接建立完成时，约经过 1.5 RTT。

不要把“客户端可以开始发送数据”和“服务器收到第三次握手”混成同一个时间点。

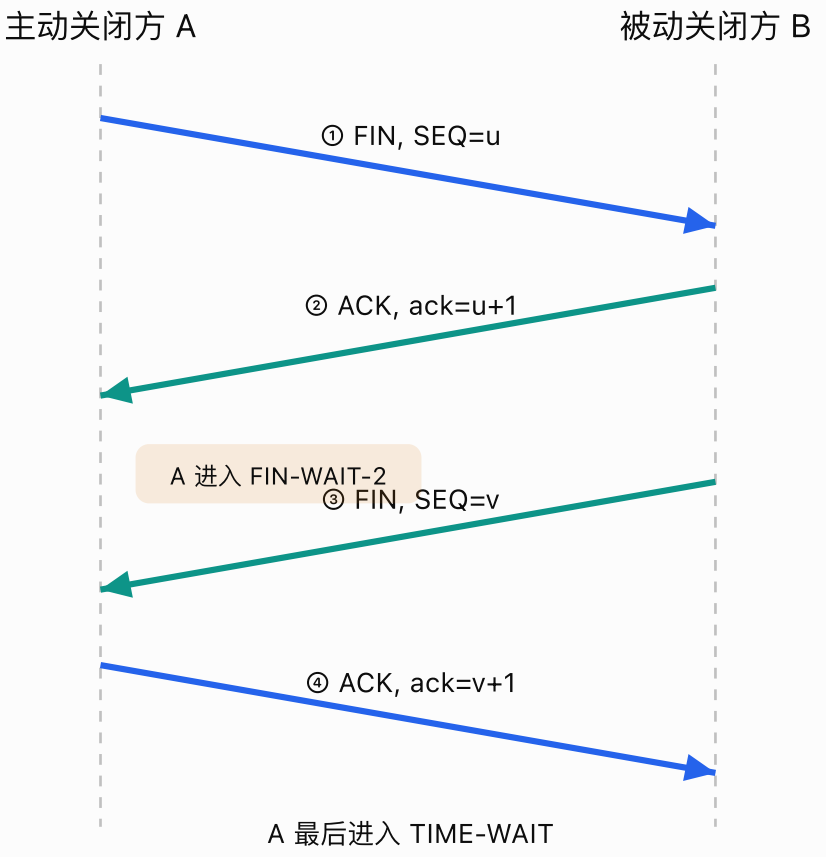
十、TCP 四次挥手（必考重点）

TCP 是全双工协议。

双方可以独立关闭各自的发送方向。

因此典型 TCP 连接释放需要四个报文段。

1. 四次挥手过程



典型四次挥手。第二次 ACK 和第三次 FIN 之间可以有时间间隔。

2. 第一次挥手

A → B：

```
纯文本
FIN = 1
SEQ = u
```

A 表示：

自己的发送方向准备关闭，不再发送新的应用数据。

A 进入：

FIN-WAIT-1

FIN 消耗一个序号。

3. 第二次挥手

B → A：

```
纯文本
ACK = 1
ack = u + 1
```

B 确认收到 A 的 FIN。

A 收到后进入：

$$\boxed{\text{FIN-WAIT-2}}$$

B 通常进入：

$$\boxed{\text{CLOSE-WAIT}}$$

此时连接处于半关闭状态。

注意：

虽然 A 已经结束发送方向，但 B 仍然可以继续发送尚未完成的数据。

4. 第三次挥手

B → A：

```
纯文本
FIN = 1
SEQ = v
```

B 也完成数据发送，准备关闭自己的发送方向。

B 进入：

$$\boxed{\text{LAST-ACK}}$$

5. 第四次挥手

A → B：

```
纯文本
ACK = 1
ack = v + 1
```

A 确认 B 的 FIN。

此时：

- B 收到最终 ACK 后关闭连接。
- A 进入 TIME-WAIT，等待规定时间后关闭。

6. 为什么有时 ACK 和 FIN 可以合并?

因为第二次挥手的 ACK 和第三次挥手的 FIN 分别代表：

- ACK：确认对方已结束发送。
- FIN：自己也结束发送。

如果 B 接收到 A 的 FIN 时，自己恰好也已经准备结束发送，就可以发送：

```
纯文本
FIN=1, ACK=1
```

将两个报文合并。

因此：

典型情况下是四次挥手，但不代表在任何情况下都一定发送四个独立报文段。

7. TIME-WAIT（重点）

主动关闭方通常在发送最终 ACK 后进入 TIME-WAIT。

经典 TCP 规定等待：

$$\boxed{2MSL}$$

MSL：Maximum Segment Lifetime，报文段最大生存时间。

其目的主要包括：

- 如果最后一个 ACK 丢失，仍能响应对方重传的 FIN。
- 让旧连接中滞留的报文段有时间消失，避免影响后续连接。

注意：TIME-WAIT 不等于等待两个 RTT。

十一、TCP 连接释放的 RTT 计算

408 很容易在这类题中设置陷阱。

假设：

- A 主动发起关闭。
- 单向传播时延为 T_p。
- RTT=$2T_p$。
- 忽略发送与处理时延。

如果 B 收到 A 的 FIN 后，立即返回 ACK，并且立刻发送 FIN：

时间	事件
0	A 发送 FIN
$0.5RTT$	B 收到 FIN，发 ACK 和 FIN
$1RTT$	A 收到 ACK 和 FIN，发最终 ACK
$1.5RTT$	B 收到最终 ACK

因此：

$$\boxed{\text{B 收到最后 ACK 的时间} = 1.5RTT}$$

但如果 B 在 ACK 之后，还需要等待一段时间 Δt 才发送 FIN，则相应的连接释放完成时间也会增加。

考试不要简单背“四次挥手需要 2 RTT”。

正确方法是：

1. 看题目具体给了哪些报文。
2. 看 ACK 和 FIN 是否合并。
3. 画出报文发送、到达的时间线。
4. 根据题目指定的结束事件计算时间。

十二、TCP 流量控制（必考重点）

流量控制解决的是：

发送方不能发送得太快，以免接收方来不及处理或缓存数据。

1. 接收窗口 rwnd

TCP 接收方通过 TCP 首部中的窗口字段，向发送方通告自己目前能够接收的数据量。

称为：

$$\boxed{rwnd}$$

接收窗口反映接收方缓存及相关接收能力约束。

例如：

接收方通告：

$$rwnd = 4000\ \mathrm{B}$$

表示发送方应按照当前通告的窗口范围限制未确认数据。

注意：TCP 首部的窗口字段原始宽度为 16 bit，但通过窗口扩大选项，实际接收窗口可以大于 65535 B。

2. 滑动窗口工作过程

假设：

- 接收窗口为 4000 B。
- 发送方当前可发送数据范围为 1000～4999。



3. 零窗口问题

如果接收方缓存暂时没有空间，可以通告：

$$\boxed{rwnd = 0}$$

发送方通常暂停发送普通新数据。

但如果接收方后来窗口增大，而窗口更新报文丢失，可能导致双方陷入等待。

TCP 使用持续计时器（Persist Timer）及零窗口探测机制帮助解决这一问题。

考点：接收窗口为 0，不代表 TCP 连接断开。

十三、TCP 拥塞控制（必考核心）

拥塞控制解决的是：

防止发送方发送的数据超过网络的承载能力，造成网络拥塞。

与流量控制的区别：

对比	流量控制	拥塞控制
主要关注	接收方是否来得及接收	网络是否发生拥塞
核心窗口	rwnd	cwnd
主要依据	接收方通告	网络反馈与拥塞算法
解决目标	保护接收方	保护网络

1. 实际发送窗口

在经典 TCP 分析中：

$$\boxed{W_{\text{send}} = \min(rwnd, cwnd)}$$

其中：

- $rwnd$：接收窗口。
- $cwnd$：拥塞窗口。

例如：

$$rwnd = 8000\ \text{B}$$

$$cwnd = 5000\ \text{B}$$

则：

$$\boxed{W_{\text{send}} = 5000\ \text{B}}$$

注意：这是总的未确认数据窗口上限。若已经有数据在途，还需减去已发送但尚未确认的数据量，才能确定当前还能发送多少新数据。

2. 拥塞窗口 cwnd

cwnd 是发送方根据网络拥塞情况维护的窗口大小。

经典教材中通常将其简化为 MSS 的整数倍。

例如：

$$cwnd = 8MSS$$

表示拥塞窗口允许的数据量为 8 个最大报文段数据长度。

3. 慢开始门限 ssthresh

慢开始门限：

$$\boxed{ssthresh}$$

用于决定经典 TCP 拥塞控制算法从慢开始向拥塞避免转换的条件。

十四、慢开始 Slow Start（必考计算）

慢开始的特点是：

拥塞窗口在每个 RTT 内大致翻倍。

前提是 ACK 正常到达，且未发生拥塞等限制。

教材经典模型：

初始：

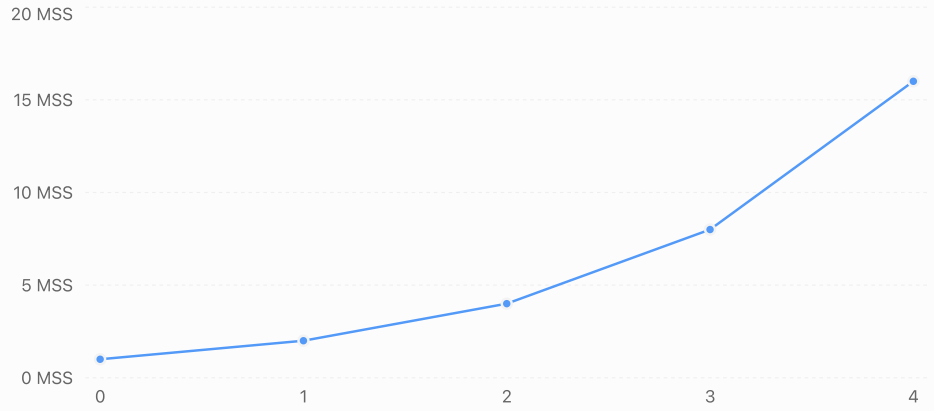
$$cwnd = 1MSS$$

随后：

$$1 \to 2 \to 4 \to 8 \to 16$$

慢开始阶段的拥塞窗口变化

经典教材简化模型：每经过一个 RTT，cwnd 翻倍



注意：慢开始并不是窗口增长缓慢。

它的增长是指数型的。

1. 典型例题

初始：

$$cwnd = 1MSS$$

假设没有拥塞、没有接收窗口限制，慢开始进行 4 个 RTT 后：

$$cwnd = 1 \times 2^4$$

$$\boxed{cwnd = 16MSS}$$

2. 为什么初始窗口不一定是 1 MSS?

现代 TCP 的初始拥塞窗口可以比 1 MSS 大。

但 408 教材和真题通常会给出明确的初始值，或者采用经典简化模型。

考试一定以题目给出的初始 cwnd 为准。

十五、拥塞避免 Congestion Avoidance（必考）

当拥塞窗口达到慢开始门限后，经典 TCP 通常进入拥塞避免阶段。

其特点是：

每经过一个 RTT，拥塞窗口大致增加 1 MSS。

例如：

$$cwnd = 8MSS$$

随后：

$$8 \to 9 \to 10 \to 11 \to 12$$

这称为加性增大。

1. 慢开始与拥塞避免对比

比较	慢开始	拥塞避免
增长方式	指数增长	线性增长
经典每 RTT 变化	约翻倍	约加 1 MSS
目的	快速探测可用带宽	更平稳地增加发送量

十六、发生拥塞后怎么办？（必考）

经典 TCP Reno 区分两种主要情况：

1. 超时重传。
2. 收到 3 个重复 ACK。

1. 超时重传

经典教材模型中，发生重传超时 RTO：

$$\boxed{ssthresh = \frac{cwnd}{2}}$$

$$\boxed{cwnd = 1MSS}$$

随后重新进入慢开始。

例：

发生超时时：

$$cwnd = 16MSS$$

则：

$$ssthresh = 8MSS$$

$$cwnd = 1MSS$$

之后：

$$1 \to 2 \to 4 \to 8$$

达到门限后进入拥塞避免：

$$8 \to 9 \to 10 \cdots$$

2. 三个重复 ACK

经典 TCP Reno 会触发：

- 快速重传。
- 快速恢复。

考试常见简化模型：

$$\boxed{ssthresh = \frac{cwnd}{2}}$$

随后进入快速恢复，避免像超时那样从 1 MSS 重新慢开始。

例如：

$$cwnd = 16MSS$$

发生 3 个重复 ACK。

门限变为：

$$ssthresh = 8MSS$$

经典 Reno 实际快速恢复过程中会临时调整 cwnd；进入正常拥塞避免时的窗口通常回到新的门限附近。

3. Tahoe 与 Reno 的区别

事件	TCP Tahoe	TCP Reno
超时	cwnd 重置到初始小窗口，慢开始	同样执行慢开始
3 个重复 ACK	快速重传后慢开始	快速重传、快速恢复
是否采用快速恢复	否	是

408 易错点：

同样是 3 个重复 ACK，Tahoe 和 Reno 的窗口变化不同。

因此题目涉及具体协议版本时，必须先看是 Tahoe 还是 Reno。

十七、拥塞窗口变化综合例题

假设采用教材简化的 TCP Reno 模型：

- 初始 $cwnd = 1MSS$。
- 初始 $ssthresh = 8MSS$。
- 未拥塞时正常收到确认。
- 在 cwnd=12 MSS 时发生超时。

求窗口变化。

第一阶段：慢开始

$$1 \to 2 \to 4 \to 8$$

第二阶段：拥塞避免

$$8 \to 9 \to 10 \to 11 \to 12$$

第三阶段：超时

门限减半：

$$ssthresh = \frac{12}{2} = 6$$

窗口重置：

$$cwnd = 1$$

第四阶段：再次增长

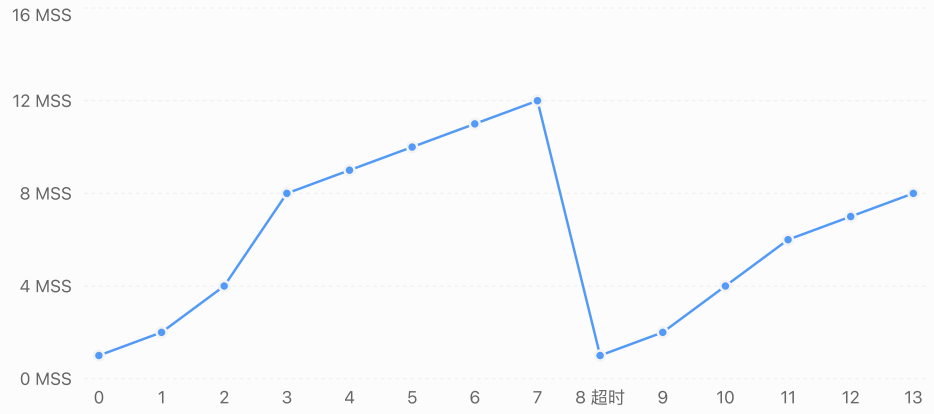
慢开始：

$$1 \to 2 \to 4$$

接近或达到门限时，按照题目规定的具体增长模型进入拥塞避免。

TCP 超时后的拥塞窗口变化

经典简化示意；超时时从 12 MSS 降到 1 MSS，新门限为 6 MSS



4. 高频判断规则

考试遇到窗口变化曲线：

- 快速翻倍 → 慢开始。
- 缓慢线性增大 → 拥塞避免。
- 突然降到 1 MSS → 经典超时处理或 Tahoe 式重新慢开始。
- 下降到原来约一半后继续线性增长 → 可能是 Reno 快速恢复完成。

记忆口诀：

慢开始指数增，拥塞避免线性增；超时回小窗口，三重 ACK 看版本。

十八、MSS、MTU 与 TCP 分段（必考）

这三个概念很容易混淆：

- MTU：最大传输单元。
- MSS：TCP 最大报文段长度。
- TCP 报文段总长度。

1. MTU

MTU 是链路能够承载的最大网络层数据报长度。

普通以太网常见：

$$\boxed{MTU = 1500\ \mathrm{B}}$$

2. MSS

MSS：Maximum Segment Size。

MSS 表示一个 TCP 报文段能够承载的最大**TCP 数据部分**长度，不包括 TCP 首部。

普通 IPv4 以太网中，如果：

- MTU=1500 B。
- IPv4 首部=20 B。
- TCP 首部=20 B。

则：

$$MSS = 1500 - 20 - 20$$

$$\boxed{MSS = 1460\ \mathrm{B}}$$

注意：1460 B 是典型数值，不代表所有 TCP 连接的 MSS 一定是 1460 B。

3. MSS 与 TCP 数据分段

例如：

应用程序需要发送：

$$4000\ \mathrm{B}$$

假设：

$$MSS = 1460\ \mathrm{B}$$

不考虑其他发送策略约束，按最大 MSS 分段：

$$4000 = 1460 + 1460 + 1080$$

所以可以分成 3 个 TCP 数据报文段。

如果初始数据序号为 1000：

报文段	数据长度	SEQ	数据序号范围
1	1460 B	1000	1000～2459
2	1460 B	2460	2460～3919
3	1080 B	3920	3920～4999

全部正确收到后：

$$\boxed{ACK = 5000}$$

解题模板：

$$\boxed{\mathrm{MSS} \rightarrow \text{分段} \rightarrow \text{计算 SEQ} \rightarrow \text{计算 ACK}}$$

4. 典型易错点

TCP 的 MSS 不包括 TCP 首部。

但 IP 数据报的总长度包括：

$$\boxed{\text{IP 首部} + \text{TCP 首部} + \text{TCP 数据}}$$

例如，TCP 数据 1000 B：

- IP 首部 20 B。
- TCP 首部 20 B。

则 IP 数据报长度：

$$20 + 20 + 1000 = 1040\ \text{B}$$

若采用普通无 VLAN 标签的 Ethernet II：

$$\text{MAC 帧长度} = 14 + 1040 + 4$$

$$\boxed{1058\ \text{B}}$$

十九、TCP 往返时间 RTT 与超时重传（重点）

1. RTT

RTT：Round-Trip Time。

指数据从发送方出发，到对应确认返回发送方所经历的往返时间。

它包含：

- 数据发送时间。
- 网络传播时间。
- 中间设备排队和处理时间。
- 确认返回时间。

在简单模型中，如果忽略其他时延：

$$\boxed{RTT \approx 2T_p}$$

但实际网络中，RTT 通常不能直接等同于两倍传播时延。

2. RTO

RTO：Retransmission Timeout。

即重传超时时间。

如果发送方在规定的 RTO 内没有获得所需确认，就可能触发超时重传。

RTO 不等于 RTT。

RTO 通常根据 RTT 的估计值及波动情况动态计算。

3. 平滑 RTT

408 可能考查经典的指数加权平均公式：

$$\boxed{RTT_s = (1-\alpha)RTT_s + \alpha RTT_{\text{sample}}}$$

这里等式右侧的 RTT_s 指更新前的估计值。

常见：

$$\alpha = \frac{1}{8}$$

例如：

- 原平滑 RTT：100 ms。
- 本次采样 RTT：140 ms。
- $\alpha = 1/8$。

则：

$$RTT_s' = \frac{7}{8} \times 100 + \frac{1}{8} \times 140$$

$$= 87.5 + 17.5$$

$$\boxed{RTT_s' = 105\ \text{ms}}$$

4. RTT 偏差与 RTO

常见估计公式：

$$\boxed{RTTVAR' = (1-\beta)RTTVAR + \beta|RTT_s - RTT_{\text{sample}}|}$$

其中：

$$\beta = \frac{1}{4}$$

这里绝对值中的 RTT_s 使用更新前的平滑 RTT。

然后：

$$\boxed{RTO = RTT_s' + 4RTTVAR'}$$

这是一种经典简化表达；实际计算还考虑时钟粒度、最小 RTO 等规则。

5. Karn 算法

重传后，某个 ACK 到来时可能无法判断它确认的是原始发送的数据，还是重传的数据。

因此：

存在重传歧义时，不直接使用该 ACK 计算新的 RTT 样本。

这就是 Karn 算法的重要思想。

二十、TCP 超时重传与快速重传

1. 超时重传

当发送的数据未在 RTO 内得到确认时，发送方重传相关数据。

特点：

- 依赖计时器。
- 无需等待重复 ACK。
- 超时通常被认为是比较明显的拥塞或丢包信号。

2. 快速重传

TCP Reno 等经典实现可以在收到 3 个重复 ACK 后，提前判断某段数据可能丢失。

不必等待 RTO 到期，就执行重传。

3. 典型例题

发送方依次发送：

报文段	SEQ	数据长度
1	1000	500 B
2	1500	500 B
3	2000	500 B
4	2500	500 B
5	3000	500 B

第 2 段丢失。

接收方已经收到第 1 段：

$$ACK = 1500$$

随后收到第 3、4、5 段，但由于第 2 段丢失，累计确认号仍然为：

$$ACK = 1500$$

因此可能连续产生 3 个重复 ACK：

```
纯文本
ACK=1500
ACK=1500
ACK=1500
```

发送方据此快速重传：

$$\boxed{SEQ = 1500}$$

即丢失的第 2 段。

注意：这属于经典重复 ACK 触发快速重传的模型；实际接收行为还可能受到延迟确认、SACK 和实现策略影响。

二十一、TCP 吞吐率与窗口计算（必考）

1. 窗口限制的吞吐率

假设：

- 发送窗口大小为 W Byte。
- RTT 为 T 秒。

在稳定流水线、持续有数据发送且无丢包等理想条件下：

$$\boxed{\text{吞吐率} \lesssim \frac{W}{T}}$$

单位为 Byte/s。

换算成 bit/s：

$$\boxed{R \lesssim \frac{8W}{RTT}}$$

实际吞吐率还受到链路带宽、协议开销、拥塞以及应用程序等条件约束。

2. 典型例题

某 TCP 连接：

- 接收窗口：64 KiB。
- RTT：20 ms。
- 拥塞窗口足够大。
- 链路容量不构成限制。

求窗口限制下的理论吞吐率。

第一步：换算窗口。

$$W = 64 \times 1024 = 65536\ \mathrm{B}$$

第二步：换算 RTT。

$$RTT = 0.02\ \mathrm{s}$$

第三步：计算。

$$R = \frac{65536 \times 8}{0.02}$$

$$= 26214400\ \mathrm{bit/s}$$

$$\boxed{R \approx 26.2\ \mathrm{Mb/s}}$$

3. 带宽时延积 BDP

BDP：Bandwidth-Delay Product。

$$\boxed{BDP = R \times RTT}$$

其中：

- R：链路带宽，单位 bit/s。
- RTT：往返时间，单位 s。

BDP 表示按该速率发送时，一个 RTT 内对应的在途数据量。

例如：

- 带宽：100 Mb/s。
- RTT：40 ms。

$$BDP = 100 \times 10^6 \times 0.04$$

$$= 4 \times 10^6\ \mathrm{bit}$$

换算字节：

$$\boxed{BDP = 500000\ \mathrm{B}}$$

因此，为了在理想情况下充分利用链路，发送窗口需要达到约 500000 B。

4. 窗口与 MSS 结合

假设：

- MSS=1000 B。
- cwnd=8 MSS。

- rwnd=6000 B。

则：

$$cwnd = 8000\ \mathrm{B}$$

实际窗口上限：

$$W = \min(8000, 6000)$$

$$\boxed{W = 6000\ \mathrm{B}}$$

如果当前已经发送并且尚未确认 4000 B，则当前最多还能发送：

$$6000 - 4000$$

$$\boxed{2000\ \mathrm{B}}$$

这个题型一定要注意区分：

发送窗口总大小与**当前还能发送的数据量**。

二十二、TCP 延迟确认与捎带确认

1. 延迟确认

接收方收到数据后，不一定立即为每一段都发送一个独立 ACK。

某些情况下可以短暂延迟确认，以减少确认报文数量。

因此：

TCP 并不要求每接收一个数据报文段，就立即独立发送一个 ACK。

2. 捎带确认

TCP 是全双工协议。

当 B 收到 A 的数据，并且自己也有数据要发送给 A 时，可以把确认信息附加在自己的 TCP 数据报文段中一起发送。

例如：

A 向 B 发送 500 B：

```
纯文本
SEQ=1000
DATA=500 B
```

B 需要确认：

$$ACK = 1500$$

如果 B 恰好也需要向 A 发送 800 B 数据，可以发送：

```
纯文本
```

```
SEQ=3000
```

这就是捎带确认。

3. ACK 标志与确认号

注意区分：

- ACK 标志位：表示确认号字段有效。
- 确认号：具体期望收到的下一个字节序号。

连接建立后的普通 TCP 报文段通常 ACK 标志为 1，即使它主要用于传输数据。

二十三、TCP 序号计算的特殊规则

普通数据段：

$$\text{下一 SEQ} = \text{当前 SEQ} + \text{数据字节数}$$

但 TCP 的 SYN、FIN 存在特殊规则。

1. SYN 消耗一个序号

例如：

```
纯文本
SYN=1
SEQ=100
```

对方应确认：

$$\boxed{ACK = 101}$$

2. FIN 消耗一个序号

例如：

```
纯文本
FIN=1
SEQ=500
```

如果 FIN 前的数据都已按序收到：

$$\boxed{ACK = 501}$$

3. 纯 ACK 不消耗序号

例如：

```
纯文本
ACK=1
SEQ=1000
DATA=0 B
```

下一次发送新数据时，仍然可以从：

$$\boxed{SEQ = 1000}$$

开始。

4. SYN、FIN 与数据同时出现

对于包含数据的 TCP 报文段，如果同时包含 SYN 或 FIN，应分别考虑其对序号空间的占用。

通用理解：

$$\boxed{\text{占用序号数} = \text{数据字节数} + \text{SYN 占位} + \text{FIN 占位}}$$

其中 SYN、FIN 每个占一个序号。

二十四、TCP 状态转换（重点）

408 不要求你像开发 TCP 协议栈那样记住所有异常分支，但以下状态必须认识。

1. 建立连接过程

状态	含义
CLOSED	连接关闭
LISTEN	服务器监听连接请求
SYN-SENT	主动连接方已发送 SYN
SYN-RCVD	已收到 SYN，并发送 SYN+ACK
ESTABLISHED	连接建立成功

典型客户端：

$$\boxed{CLOSED \rightarrow SYN\text{-}SENT \rightarrow ESTABLISHED}$$

典型服务器：

$$\boxed{LISTEN \rightarrow SYN\text{-}RCVD \rightarrow ESTABLISHED}$$

2. 释放连接过程

主动关闭方：

$$\boxed{ESTABLISHED \rightarrow FIN\text{-}WAIT\text{-}1 \rightarrow FIN\text{-}WAIT\text{-}2 \rightarrow TIME\text{-}WAIT \rightarrow CLOSED}$$

被动关闭方：

$$\boxed{ESTABLISHED \rightarrow CLOSE\text{-}WAIT \rightarrow LAST\text{-}ACK \rightarrow CLOSED}$$

这是典型的四次挥手路径，TCP 也存在同时关闭等其他情况。

3. 高频状态判断

- 收到 SYN，回复 SYN+ACK 后 → SYN-RCVD。
- 主动关闭方收到对 FIN 的单独确认后 → FIN-WAIT-2。
- 被动关闭方收到 FIN、确认后 → CLOSE-WAIT。
- 被动关闭方发送自己的 FIN 后 → LAST-ACK。
- 主动关闭方收到对方 FIN、发送最后 ACK 后 → TIME-WAIT。

二十五、TCP 与 UDP 的典型应用题

1. 哪些应用适合 TCP?

通常适合：

- 需要可靠字节流的文件传输。
- SSH 远程登录。
- 电子邮件传输。
- HTTP/1.1、HTTP/2。

2. 哪些应用适合 UDP?

UDP 常用于：

- DNS 查询。
- DHCP。
- 实时音视频。
- 部分在线通信协议。
- QUIC。

需要注意：UDP 本身不提供可靠交付，但运行在 UDP 上的应用层或其他上层协议可以自己实现确认、重传、拥塞控制等机制。

因此不能认为：

UDP 上运行的应用一定不可靠。

例如，QUIC 可以在 UDP 之上实现可靠传输。

二十六、408 传输层常考综合题型

题目关键词	对应知识点	解题方法
端口、四元组	复用与分用	看 IP、端口组合
UDP 数据长度	UDP 首部	数据 + 8 B
TCP 数据偏移	TCP 首部	字段值 × 4 B
SEQ、数据长度	TCP 序号	求下一个字节序号
ACK、丢失数据	累积确认	找第一个缺失字节
SYN、FIN	序号空间	各消耗 1 个序号
三次握手	连接建立	SYN → SYN+ACK → ACK
四次挥手	连接释放	FIN → ACK → FIN → ACK
rwnd、cwnd	发送窗口	取较小值
ssthresh	慢开始与拥塞避免	判断增长阶段
3 个重复 ACK	快速重传	根据协议版本判断窗口
RTO	超时重传	重传并调整拥塞控制
RTT、窗口	吞吐率	$8W/RTT$
MSS、MTU	TCP 分段	扣除 IP、TCP 首部

二十七、综合例题一：TCP 序号与确认号

假设 TCP 连接已经建立。

A 向 B 发送三个报文段：

报文段	SEQ	数据长度
1	1000	500 B
2	1500	500 B
3	2000	500 B

其中第 2 段丢失。

第一问：B 收到第 1 段后，确认号是多少?

第一段序号范围：

$$1000 \sim 1499$$

因此：

$$\boxed{ACK = 1500}$$

第二问：B 随后收到第 3 段，确认号是多少?

由于 1500～1999 仍然缺失：

$$\boxed{ACK = 1500}$$

第三问：第 2 段重传并正确收到后，确认号是多少?

假设第 3 段已被接收方正确缓存。

现在：

- 1000～1499 已收到。
- 1500～1999 已收到。
- 2000～2499 已收到。

因此：

$$\boxed{ACK = 2500}$$

这类题的关键不是“最后收到哪一个报文段”，而是：

从当前连接的字节流起点开始，已经连续收到哪里。

二十八、综合例题二：拥塞窗口变化

某 TCP Reno 连接按照教材简化模型运行：

- 初始 cwnd=1 MSS。
- ssthresh=16 MSS。
- 每 RTT 正常增长。
- cwnd 达到 20 MSS 时发生超时。

求各阶段变化。

第一阶段：慢开始

$$1 \to 2 \to 4 \to 8 \to 16$$

第二阶段：拥塞避免

$$16 \to 17 \to 18 \to 19 \to 20$$

第三阶段：发生超时

$$ssthresh = \frac{20}{2} = 10$$

$$cwnd = 1$$

第四阶段：重新增长

$$1 \to 2 \to 4 \to 8$$

接近新的门限后，按题目规定的具体规则切换到拥塞避免，继续线性增长。

高频易错点：

超时之后的 ssthresh 不是保持原来的 16 MSS，而是根据发生超时时的拥塞窗口重新确定。

二十九、综合例题三：发送窗口与实际可发送量

设：

$$rwnd = 12000\ \mathrm{B}$$

$$cwnd = 8000\ \mathrm{B}$$

已经发送但尚未确认：

$$5000\ \mathrm{B}$$

问现在最多还能发送多少新数据?

第一步：

$$W = \min(12000, 8000) = 8000$$

第二步：减去已经在途的数据。

$$8000 - 5000 = 3000$$

所以：

$$\boxed{3000\ \mathrm{B}}$$

如果题目进一步告诉你：

$$MSS = 1000\ \mathrm{B}$$

那么在不考虑其他限制时，最多还能发送三个满 MSS 的数据段。

三十、传输层易错判断练习

传输层自测 0/12 已答

1. TCP 面向字节流，所以保留每次应用层写入的消息边界。
2. UDP 首部长度固定为 8 B。
3. TCP 序号通常指报文段数据部分第一个字节的序号。
4. TCP 确认号表示最后一个已经正确收到的字节序号。
5. 一个不携带数据的纯 ACK 通常不占用 TCP 序号。
6. TCP 连接必须通过三次握手建立。
7. TCP 四次挥手中，第二次 ACK 和第三次 FIN 在任何情况下都必须分开发送。
8. TCP 的流量控制主要防止发送方超过接收方的处理和缓存能力。

9. 在经典模型中，TCP 的实际发送窗口上限取 rwnd 与 cwnd 中较小者。

10. TCP 慢开始阶段，cwnd 通常每经过一个 RTT 增加 1 MSS。

11. TCP Reno 在发生超时后通常重新进入慢开始。

12. MSS 包括 TCP 首部和 TCP 数据。